Vishal Acharya

Module

A Python module is the highest level of program organization, packing code, and data for reuse and easy interoperability at the functional level and for providing self-contained namespaces to prevent variables from clashing across different programs.

Modules are files containing Python code that define functions, classes, and variables. Runnable code can also be included in a module. These pieces should be shared, so Python allows a module to "bring in" and use attributes from other modules to take advantage of work that has been done, maximizing code reusability. This process of associating the attributes from other modules with your module is called importing.

INTRODUCTION

A module is a file containing Python definitions and statements.

A module is a file containing group of variables, methods, function and classes etc.

They are executed only the first time the module name is encountered in an import statement.

The file name is the module name with the suffix .py appended.

Ex:- mymodule.py

Type of Modules:-

User-defined Modules

Built-in Modules Ex:- array, math, sys

When and Why use Module

Assume that you are building a very large project, it will be very difficult to manage all logic within one single file so If you want to separate your similar logic to a separate file, you can use module.

It will not only separate your logics but also help you to debug your code easily as you know which logic is defined in which module. When a module is developed, it can be reused in any program that needs that module.

Vishal Acharya

We will cover the following topics in this chapter:

The import statement

The from..import statement

The from..import * statement

Executing modules as scripts

How to use Module

import statement is used to import modules.

Syntax:-

import module_name

import module_name as alias_name

from module_name import

from module_name import f_name as alias_f_name

from module_name import *

from module_name import class_name1, class_name2,....., class_nameN

from module_name import var_name1, var_name2,....., var_nameN

from module_name import function_name1, function_name2,....., function_nameN

from module_name import var_name, f_name, class_name, method_name.....,

The import statement

Any Python source file can be used as a module by executing an import statement in another Python source file.

Syntax:

import module1[, module2[,... moduleN]

When the interpreter finds an import statement, it imports the module if it is present in the search path.

Search The search path refers to a list of directories that the interpreter searches before importing a module

import module_name

This does not enter the names of the functions defined in module directly in the current symbol table; it only enters the module name there.

When 2 modules having same function name then This import module is good approach to use.

Ex:- import cal

How to access Methods, Functions, Variable and Classes ?

Using the module name you can access the functions.

Syntax:- module_name.function_name()

Ex:-

cal.add(10, 20)

cal.sub(20, 10)

add = cal.add

add(10, 20)

import module_name as alias_name

This does not enter the names of the functions defined in module directly in the current symbol table; it only enters the module name there. If the module name is followed by as, then the name following as is bound directly to the imported module.

Vishal Acharya

Ex:- import cal as c

How to access Methods, Functions, Variable and Classes ?

Using the alias_name you can access the functions.

Ex:- c.add(10, 20)

c.sub(20, 10)

add = c.add

add(10, 20)

The from...import statement

You can import specific attributes from a module into the current namespace using Python's from statement.

Syntax:

from modname import name1[, name2[, ... nameN]]

from module_name import function_name

There is a variant of the import statement that imports names from a module directly into the importing module's symbol table.

Ex:- from cal import add, sub

How to access Methods, Functions, Variable and Classes ?

You can access the functions directly by it's name.

Ex:-

add(10, 20)

sub(20, 10)

from module_name import f_name as a_name

Ex:- from cal import add as s

How to access Methods, Functions, Variable and Classes ? You can access the functions directly by it's alias name.

Vishal Acharya

Ex:- s(10, 20)

The from...import * statement

All the names in a module can be imported into the current namespace using the following import statement.

Syntax:

from modname import *

A module can be imported into the current namespace using this statement, but the information should be used sparingly.

from module_name import *

This imports all names except those beginning with an underscore (_).

Ex:- from cal import *

How to access Methods, Functions, Variable and Classes ?

You can access the functions directly by it's name.

Ex:-

add(10, 20)

sub(20, 10)

Executing modules as scripts

Module names (as strings) are available as the value of a global variable **name** within a module. The code in the module is executed as you imported it, but the **name** will be set to Consider that you run a Python module with

The module search path

The interpreter first searches for the built-in module named spam when importing a module named spam. If spam.py is not found, the sys.path variable is used to search a list of directories for a file named These locations are used to initialize

The directory that contains the input script (or the current directory)

This is the Pythonpath (a list of directory names with the same

Vishal Acharya

syntax as the shell variable PATH)

Installation-dependent defaults

Python programs can modify sys.path during initialization. The directory containing the script being run is placed at the start of the search path, before the standard library path. This means when scripts are loaded from that directory, modules from the library directory of the same name are not loaded. Unless the replacement is intended, this is an error.

Note: Here is a typical PYTHONPATH from a Windows system.

In [44]:
```
%%writefile cal.py
a = 50
def name():
    print("From Module cal")
def add(a,b):
    return a+b
def sub(a,b):
    return a-b
```

```
Writing cal.py
```

In [45]:
```
import cal # Importing Cal Module

print("cal Module's variable:", cal.a) # Accessing Cal Module's Variable

cal.name() #Accessing Cal Module's Function

a = cal.add(10,20) # Accessing Cal Module's Function
print(a)

b = cal.sub(20, 10) # Accessing Cal Module's Function
print(b)
```

```
cal Module's variable: 50
From Module cal
30
10
```

In [46]:
```
import cal # Importing Cal Module

print("cal Module's variable:", cal.a) # Accessing Cal Module's Variable

cal.name() # Accessing Cal Module's Function

add = cal.add # Accessing and Assigning Module's Function to Variable
a = add(10, 20) # Accessing Cal Module's Function
print(a)

b = cal.sub(20, 10) # Accessing Cal Module's Function
print(b)
```

```
cal Module's variable: 50
From Module cal
30
10
```

Vishal Acharya

Vishal Acharya

In [47]:
```
import cal as c # Importing Cal Module

print("cal Module's variable:", c.a) # Accessing Cal Module's Variable

c.name() # Accessing Cal Module's Function

a = c.add(10,20) # Accessing Cal Module's Function
print(a)

b = c.sub(20, 10) # Accessing Cal Module's Function
print(b)
```

```
cal Module's variable: 50
From Module cal
30
10
```

In [48]:
```
from cal import a, name, add, sub # Importing Cal Module

print("cal Module's variable:", a) # Accessing Cal Module's Variable

name() # Accessing Cal Module's Function

a = add(10,20) # Accessing Cal Module's Function
print(a)

b = sub(20, 10) # Accessing Cal Module's Function
print(b)
```

```
cal Module's variable: 50
From Module cal
30
10
```

In [49]:
```
from cal import a, name, add as s, sub # Importing Cal Module

print("cal Module's variable:", a) # Accessing Cal Module's Variable

name() # Accessing Cal Module's Function

add = s(10,20) # Accessing Cal Module's Function
print(add)

b = sub(20, 10) # Accessing Cal Module's Function
print(b)
```

```
cal Module's variable: 50
From Module cal
30
10
```

In [50]:
```
from cal import * # Importing Cal Module

print("cal Module's variable:", a) # Accessing Cal Module's Variable

name() # Accessing Cal Module's Function

a = add(10,20) # Accessing Cal Module's Function
print(a)
```

```
b = sub(20, 10) # Accessing Cal Module's Function
print(b)
```

```
cal Module's variable: 50
From Module cal
30
10
```

In [51]:
```
%%writefile first.py
a=50
def name():
    print("in")
```

```
Writing first.py
```

In [52]:
```
%%writefile second.py
a=40
def sname():
    print("out")
```

```
Writing second.py
```

In [53]:
```
 #Importing Two Modules and accessing their members
# Both Modules has same name variable and function
import first # Importing first Module
import second # Importing second Module

print(first.a) # Accessing first Module's Variable
first.name() # Accessing first Module's Function

print(second.a) # Accessing Second Module's Variable
second.sname() # Accessing Second Module's Function
```

```
50
in
40
out
```

In [54]:
```
%%writefile first1.py
a=50
def name():
    print("in")
```

```
Writing first1.py
```

In [55]:
```
%%writefile second1.py
a=40
def sname():
    print("out")
```

```
Writing second1.py
```

In [57]:
```
import first1 as f # Importing first Module
import second1 as s # Importing second Module

print(f.a) # Accessing first Module's Variable
f.name() # Accessing first Module's Function

print(s.a) # Accessing Second Module's Variable
s.sname() # Accessing Second Module's Function
```

Vishal Acharya

```
50
in
40
out
```

In [59]:
```
from first1 import name, a # Importing first Module

print(a) # Accessing first Module's Variable
name() # Accessing first Module's Function

from second1 import sname, a # Importing second Module

print(a) # Accessing Second Module's Variable
name() # Accessing Second Module's Function
```

```
50
in
40
in
```

In [65]:
```
%%writefile firstclass.py
class Myclass:
    def name(self):
        print("Name Method from first Module")

class Myschool:
    def show(self):
        print("Show Method from first Module")
```

```
Overwriting firstclass.py
```

In [61]:
```
import firstclass # Importing first Module

c = firstclass.Myclass() # Creating Myclass Object
c.name()

s = firstclass.Myschool() # Creating Myschool Object
s.show()
```

```
Name Method from first Module
Show Method from first Module
```

In [63]:
```
%%writefile secondclass.py
class Mycollege:
    def disp(self):
        print("Disp Method from Second Module")
```

```
Writing secondclass.py
```

In [64]:
```
import firstclass # Importing first Module
import secondclass # Importing second Module

c = firstclass.Myclass() # Creating Myclass Object - first Module
c.name()

s = firstclass.Myschool() # Creating Myschool Object - first Module
s.show()

cl = secondclass.Mycollege()  # Creating Myschool Object - second Module
cl.disp()
```

Vishal Acharya

```
Name Method from first Module
Show Method from first Module
Disp Method from Second Module
```

In [67]:
```
from firstclass import Myclass, Myschool # Importing first Module
c = Myclass() # Creating Myclass Object - first Module
c.name()

s = Myschool() # Creating Myschool Object - first Module
s.show()
from second1 import sname, a # Importing second Module

print(a) # Accessing Second Module's Variable
name() # Accessing Second Module's Function
```

```
Name Method from first Module
Show Method from first Module
40
in
```

Python - OS Module

It is possible to automatically perform many operating system tasks. The OS module in Python provides functions for creating and removing a directory (folder), fetching its contents, changing and identifying the current directory, etc.

You first need to import the os module to interact with the underlying operating system. So, import it using the import os statement before using its functions.

Handling the Current Working Directory

Consider Current Working Directory(CWD) as a folder, where the Python is operating. Whenever the files are called only by their name, Python assumes that it starts in the CWD which means that name-only reference will be successful only if the file is in the Python's CWD.

Note: The folder where the Python script is running is known as the Current Directory. This is not the path where the Python script is located. Getting the Current working directory

To get the location of the current working directory **os.getcwd()** is used

In [70]:
```
pwd
```

Out[70]:
```
'C:\\'
```

In [71]:
```
#The getcwd() function confirms returns the current working directory.
import os
os.getcwd()
```

Out[71]:
```
'C:\\'
```

Vishal Acharya

Vishal Acharya

Creating a Directory

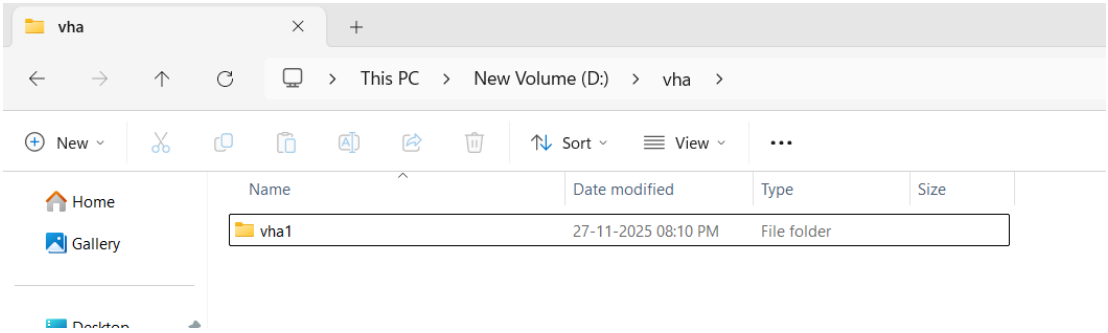
os.mkdir()

os.makedirs()

Using os.mkdir()

os.mkdir() method in Python is used to create a directory named path with the specified numeric mode. This method raises FileExistsError if the directory to be created already exists.

In [72]:
```
import os
os.mkdir(r"d:\\vha\\vha1")
```



Changing the Current working directory

To change the current working directory(CWD) os.chdir() method is used. This method changes the CWD to a specified path. It only takes a single argument as a new directory path.

Note: The current working directory is the folder in which the Python script is operating.

In order to set the current directory to the parent directory use ".." as the argument in the chdir() function.

In [73]:
```
os.chdir("d:/vha/vha1")
```

In [74]:
```
os.getcwd()
```

Out[74]:
```
'd:\\vha\\vha1'
```

In [75]:
```
os.chdir("..")
```

In [76]:
```
os.getcwd()
```

Out[76]:
```
'd:\\vha'
```

In [77]:
```
os.chdir("..")
```

In [78]:
```
os.getcwd()
```

Out[78]:
```
'd:\\'
```

Listing out Files and Directories with Python

os.listdir() method in Python is used to get the list of all files and directories in the specified directory. If we don't specify any directory, then the list of files and directories in the current working directory will be returned.

In [79]:
```
import os
os.listdir("c:")
```

Out[79]:
```
['$Recycle.Bin',
 '$SysReset',
 'Config.Msi',
 'Documents and Settings',
 'DumpStack.log',
 'DumpStack.log.tmp',
 'hiberfil.sys',
 'hp',
 'inetpub',
 'logUploaderSettings.ini',
 'logUploaderSettings_temp.ini',
 'M1130MFP_M1210MFP_Full_Solution',
 'msdia80.dll',
 'OneDriveTemp',
 'pagefile.sys',
 'PerfLogs',
 'Program Files',
 'Program Files (x86)',
 'ProgramData',
 'Recovery',
 'swapfile.sys',
 'SWSetup',
 'System Volume Information',
 'System.sav',
 'Users',
 'Windows']
```

In [80]:
```
import os
os.listdir(r"d://vha")
```

Out[80]:
```
['vha1']
```

In [81]:
```
import os
os.listdir("d:/2025_26_python_1/")
```

Vishal Acharya

Out[81]:
```
['exam',
 'EXAM_Daily_UG_SYALL_41_06 10 2025 to 11 10 2025_06 Oct 25.xlsx',
 'for i in range.docx',
 'PATTERN_VHA.ipynb',
 'PATTERN_VHA.pdf',
 'pb.pdf',
 'PB_Python-I_SEM III_2025.pdf',
 'PB_Python-I_SEM III_2025.xlsx',
 'REM Solution.ipynb',
 'shift change',
 'systems_20241231.csv',
 'T1_CLASS PROGRAM_VISHAL ACHARYA.ipynb',
 'T1_CLASS PROGRAM_VISHAL ACHARYA.pdf',
 'T1_MCQ Explanation_VHA.pdf',
 'T2',
 'T3_2025_26_VHA',
 'test 1.docx',
 'test 2.pdf',
 'Unit-1_VHA.ipynb',
 'Unit-1_VHA.pdf',
 'UNIT-3_QB_SOLUTION_VHA.ipynb',
 'UNIT-3_QB_SOLUTION_VHA.pdf',
 'UNIT_1_QB_VHA.ipynb',
 'UNIT_1_QB_VHA.pdf',
 'UNIT_2_QB_SOLUTION_VHA.ipynb',
 'UNIT_2_QB_SOLUTION_VHA.pdf',
 'Unit_2_VHA.ipynb',
 'Unit_2_VHA.pdf',
 'unit_3_VHA.pdf']
```

Vishal Acharya

Deleting Directory or Files using Python

OS module proves different methods for removing directories and files in Python. These are –

Using os.remove()

Using os.rmdir()

Using os.remove()

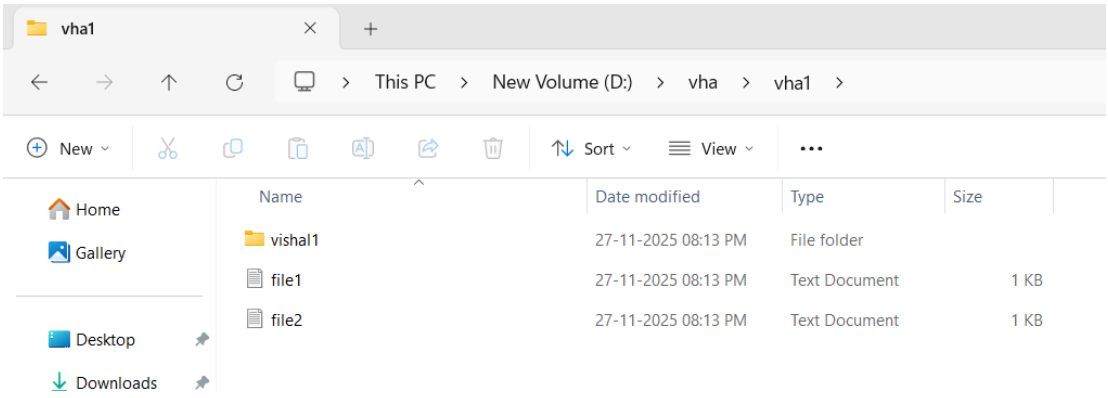
os.remove() method in Python is used to remove or delete a file path. This method can not remove or delete a directory. If the specified path is a directory then OSError will be raised by the method.

Using os.rmdir()

os.rmdir() method in Python is used to remove or delete an empty directory. OSError will be raised if the specified path is not an empty directory.

In [82]:
```
import os
os.chdir("d:/vha/vha1")
os.mkdir("vishal1")
f=open("file1.txt","w")
f.write("hello")
```

```
f.close()
f=open("file2.txt","w")
f.write("hello1")
f.close()
```



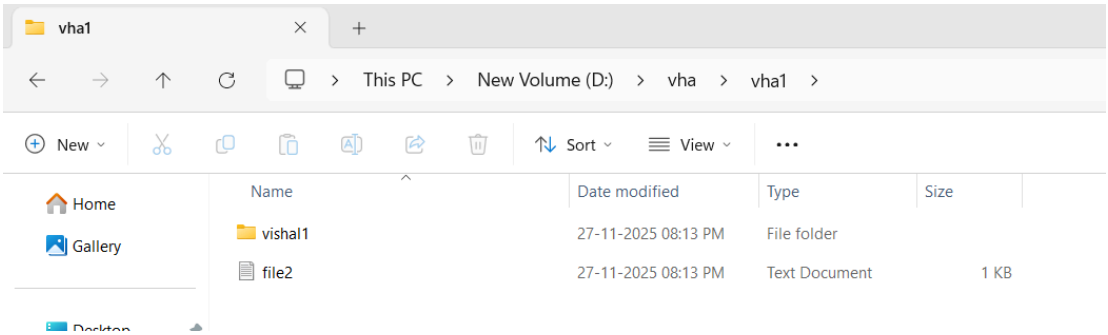
In [84]:
```
os.listdir("d:/vha")
```

Out[84]:
```
['vha1']
```

In [85]:
```
os.listdir("d:/vha/vha1")
```

Out[85]:
```
['file1.txt', 'file2.txt', 'vishal1']
```

In [86]:
```
os.remove("d:/vha/vha1/file1.txt")
```



In [88]:
```
os.rmdir("d:/vha/vha1")
```

```
---------------------------------------------------------------------------
PermissionError                           Traceback (most recent call last)
Cell In[88], line 1
----> 1 os.rmdir("d:/vha/vha1")

PermissionError: [WinError 32] The process cannot access the file because it is b
eing used by another process: 'd:/vha/vha1'
```

In [89]:
```
os.rmdir("d:/vha/vha1/vishal1")
```

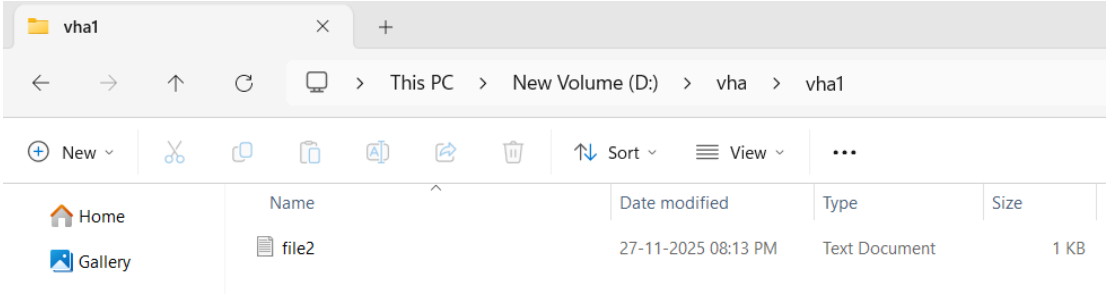
In [90]:
```
os.listdir("d:/vha")
```

Out[90]:
```
['vha1']
```

In [91]:
```
os.listdir("d:/vha/vha1")
```

Out[91]:
```
['file2.txt']
```

Vishal Acharya



In [ ]:

Vishal Acharya